Data Management VO

06 Physical Design and Tuning

Lucas Iacono

Know-Center & Graz University of Technology, Austria
Institute of Human-Centred Computing

Course calendar

- **Part 2: Physical design, query processing, APIs, and NoSQL**
  - **(6) Mon. 27.04.2025: Physical design and tuning**
  - No lecture on 04.05.2026 (time for Ex 2 + **VO exam #2**)
  - (7) Mon. 11.05.2026: **Database APIs + Python + Exercise 3**

  - No lecture on 18.05.2026 (time for Ex 3)

  - Mon. 25.05.2026: Whit Monday (public holiday, means no lecture)

  - (8) Mon. 01.06.2025: NoSQL + Data Spaces

  - No lecture on 08.06.2026 (time for Ex 3)

  - **VO exam #3** on 16.06.2025 (means no lecture)

Last lecture

- Relational algebra
  - Use algebraic operations for defining queries on data
  - Better understand how SQL works

Last lecture

- Relational algebra
  - Use algebraic operations for defining queries on data
  - Better understand how SQL works

- Query optimization
  - Logical query trees
  - Analyze + explain costs of query plans

Last lecture

- Relational algebra
  - Use algebraic operations for defining queries on data
  - Better understand how SQL works
- Query optimization
  - Logical query trees
  - Analyze + explain costs of query plans
- Submission of exercise 2
- Questions or comments?
  - See also Discord

This week: Agenda

- Physical Design and Tuning
  - What is happening in the background / on the physical DB layer?
  - Focus: index structures, b-trees

This week: Agenda

- Physical Design and Tuning
  - What is happening in the background / on the physical DB layer?
  - Focus: index structures, b-trees
- Transaction Processing
  - How do we deal with concurrency issues?
  - Just a little bit (more in Data Integration and Large-Scale Analysis)

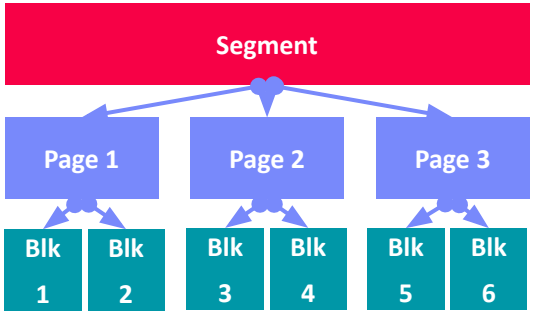
Physical design and tuning: Overview

- Compression Techniques
- **Index Structures & b-trees**
- Materialized Views

Compression Techniques

Background Storage System

- **Segments, Pages, Blocks**
  - **Segments:** storage unit of DB objects like relations and indexes
  - **Page:** fixed-size memory region
    - e.g., 8KB in PostgreSQL (default value)
  - **Block:** smallest addressable unit on disk

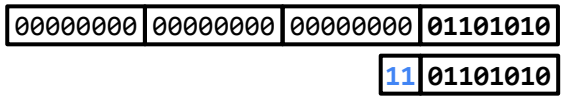


- **Compression Overview**
  - **Fit larger datasets in memory** → less I/O, more efficient caching
    - e.g., more efficient storage within the blocks
  - Sometimes query processing can directly happen **on the compressed data**

Database Compression Schemes

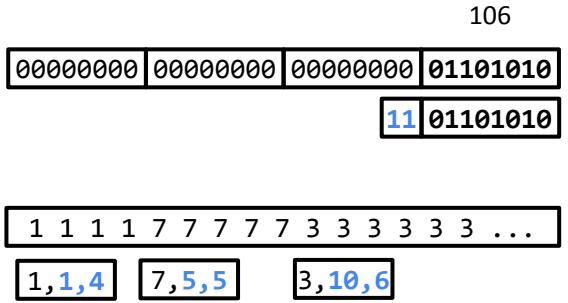
- **Null Suppression**
  - Compress integers by **omitting leading zero** bytes/bits

106



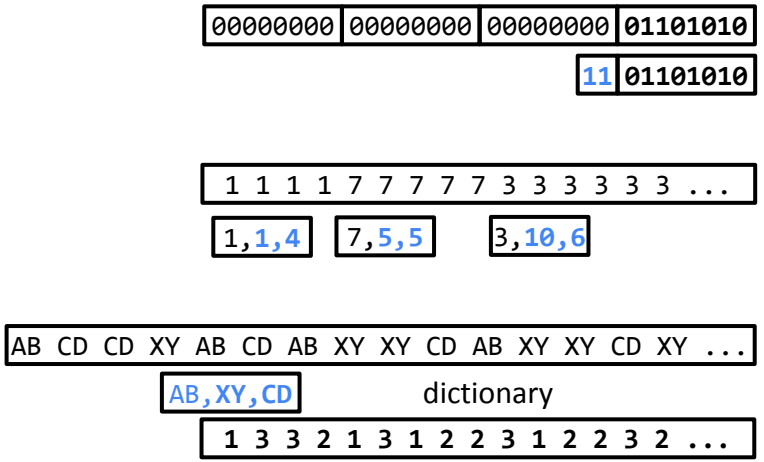
Database Compression Schemes

- **Null Suppression**
  - Compress integers by **omitting leading zero** bytes/bits
- **Run-Length Encoding**
  - Compress sequences of equal values by **runs** of (value, start, run length)



Database Compression Schemes

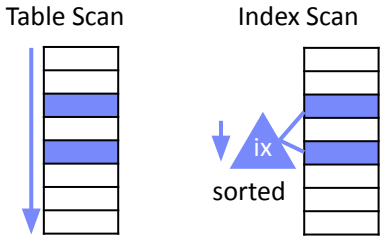
- **Null Suppression**
  - Compress integers by **omitting leading zero** bytes/bits
- **Run-Length Encoding**
  - Compress sequences of equal values by **runs** of (value, start, run length)
- **Dictionary Encoding**
  - Compression for few distinct values as **position in dictionary**



Index Structures
and
b-trees

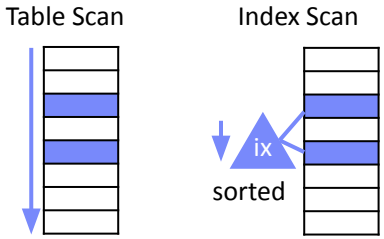
Overview Index Structures

- **Table Scan vs Index Scan**
  - ix could be an index of column sorted in a specific way
  - For highly selective predicates, **index scan is much better** than **table scan**
  - Especially for **queries referencing only few tuples**
  - E.g., get courses of student with id = 500 (WHERE S.ID = 500)



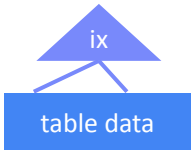
Overview Index Structures

- **Table Scan vs Index Scan**
  - ix could be an index of column sorted in a specific way
  - For highly selective predicates, **index scan is much better** than **table scan**
  - Especially for **queries referencing only few tuples**
  - E.g., get courses of student with id = 500 (WHERE S.ID = 500)



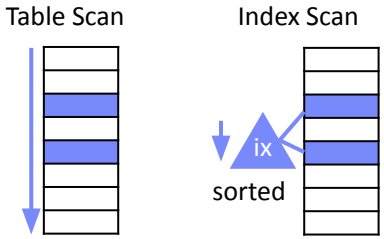
- **Use Cases for Indexes**

Lookups



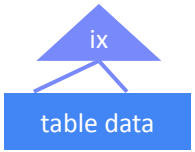
Overview Index Structures

- **Table Scan vs Index Scan**
  - ix could be an index of column sorted in a specific way
  - For highly selective predicates, index scan **is much better** than table scan
  - Especially for queries referencing only few tuples
  - E.g., get courses of student with id = 500

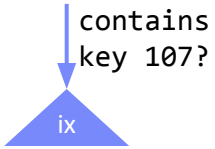


- **Use Cases for Indexes**

Lookups

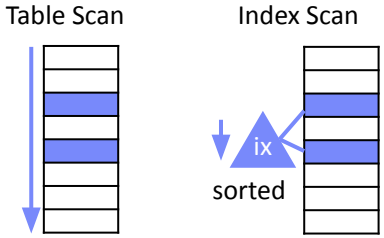


Unique Constraints



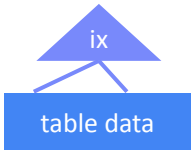
Overview Index Structures

- **Table Scan vs Index Scan**
  - ix could be an index of column sorted in a specific way
  - For highly selective predicates, index scan **is much better** than table scan
  - Especially for queries referencing only few tuples
  - E.g., get courses of student with id = 500

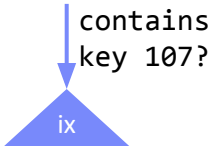


- **Use Cases for Indexes**

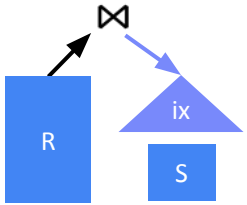
Lookups



Unique Constraints

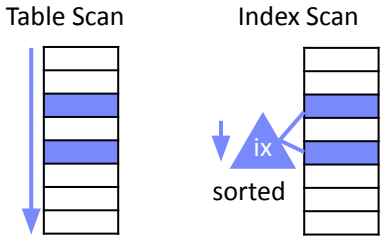


Joins



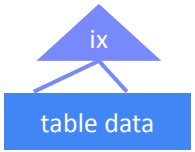
Overview Index Structures

- **Table Scan vs Index Scan**
  - ix could be an index of column sorted in a specific way
  - For highly selective predicates, index scan **is much better** than table scan
  - Especially for queries referencing only few tuples
  - E.g., get courses of student with id = 500

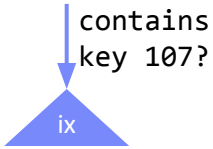


- **Use Cases for Indexes**

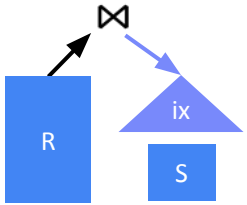
Lookups



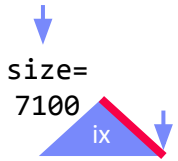
Unique Constraints



Joins



Aggregates (count, min/max)



Additional Terminology

- **Create Index**
  - **Clustered:** tuples sorted by index
    - By default created for primary key (**only one!**)
  - **Non-clustered:** sorted attribute with tuple references
    - Multiple are possible

Additional Terminology

- **Create Index**
  - **Clustered:** tuples sorted by index
    - By default created for primary key (only one!)
  - **Non-clustered:** sorted attribute with tuple references
    - Multiple are possible
  - **How to create a non-clustered** index on a set of attributes
  - Can specify attributes, order, and indexing method

```
CREATE INDEX ixStudLname
  ON Students USING btree
  (Lname ASC NULLS FIRST);
```

Additional Terminology

- **Create Index**
  - **Clustered:** tuples sorted by index
    - By default created for primary key (only one!)
  - **Non-clustered:** sorted attribute with tuple references
    - Multiple are possible
  - **How to create a non-clustered index** on a set of attributes
  - Can specify attributes, order, and indexing method
  - **How to choose attributes?**
    - Work with **analyze/explain** and see **differences in total costs** (see last lecture)
    - Index on **attributes used in selective predicates** (e.g., where)
    - Prefer **attributes with many unique values** (e.g., creation timestamp)
    - Multiple attributes per index possible, but **use one if possible**

```
CREATE INDEX ixStudLname
  ON Students USING btree
  (Lname ASC NULLS FIRST);

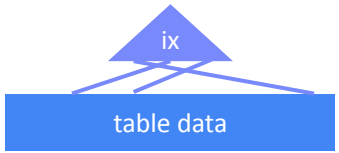
DROP INDEX ixStudLname;
```

Additional Terminology

- **Create Index**
  - **Clustered:** tuples sorted by index
    - By default created for primary key (only one!)
  - **Non-clustered:** sorted attribute with tuple references
    - Multiple are possible
  - **How to create a non-clustered index** on a set of attributes
  - Can specify attributes, order, and indexing method
  - **How to choose attributes?**
    - Work with analyze/explain and see differences in total costs (see last lecture)
    - Index on attributes used in selective predicates (e.g., where)
    - Prefer attributes with many unique values (e.g., creation timestamp)
    - Multiple attributes per index possible, but use one if possible
  - **PostgreSQL methods:** e.g., **btree** (default in PostgreSQL), hash

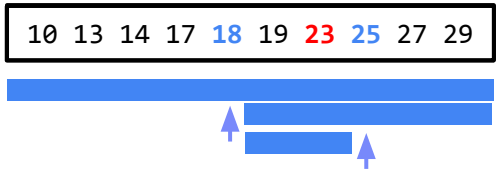
```
CREATE INDEX ixStudLname
  ON Students USING btree
  (Lname ASC NULLS FIRST);

DROP INDEX ixStudLname;
```



Prerequiste for b-trees: Binary search

- **Binary Search**
  - `pos = binarySearch(data,key=23)`
  - Given **sorted data**, find key position (insert position if non-existing)
  - Split data in half
    - If smaller left (exclude right)
    - If larger right (exclude left)
    - Recursive until found (or not)



B-Tree Overview

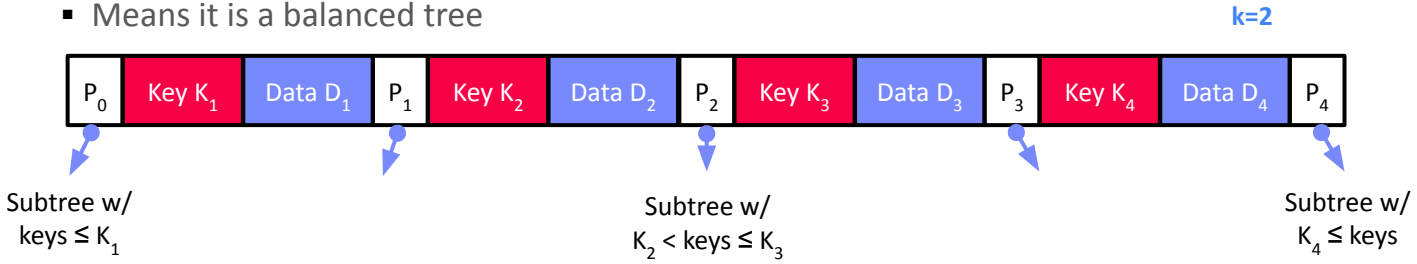
- **Definition B-Tree (k, h)**
  - Nodes have entries consisting of keys, data and pointers
    - Data is a record or a reference to the record (RID)
    - Keys are sorted □ binary search to find keys
    - Pointers point to successors (subtrees/leafs)

B-Tree Overview

- **Definition B-Tree (k, h)**
  - Nodes have entries consisting of keys, data and pointers
    - Data is a record or a reference to the record (RID)
    - Keys are sorted □ binary search to find keys
    - Pointers point to successors (subtrees/leafs)
  - All nodes (except root) have **[k, 2k]** keys
  - All nodes (except root, leafs) have **[k+1, 2k+1]** successors
  - All paths from root to leafs have equal length h (= height of tree)
    - Means it is a balanced tree

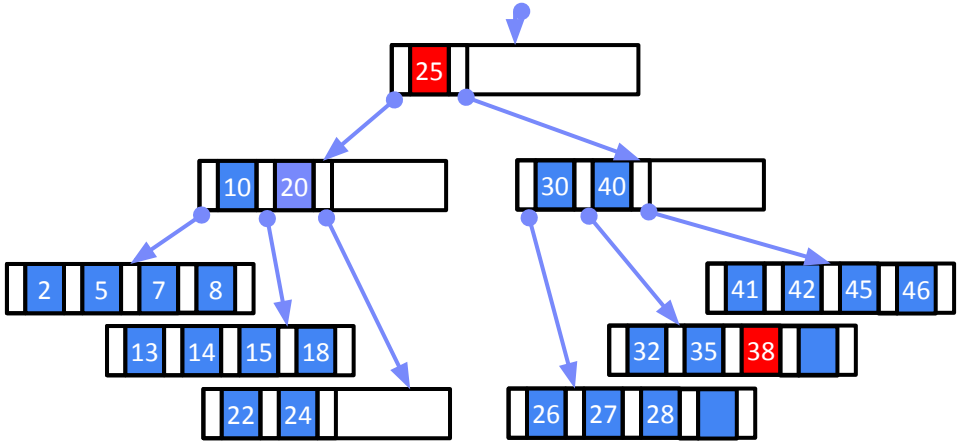
B-Tree Overview

- **Definition B-Tree (k, h)**
  - Nodes have entries consisting of keys, data and pointers
    - Data is a record or a reference to the record (RID)
    - Keys are sorted □ binary search to find keys
    - Pointers point to successors (subtrees/leafs)
  - All nodes (except root) have **[k, 2k]** keys
  - All nodes (except root, leafs) have **[k+1, 2k+1]** successors
  - All paths from root to leafs have equal length h (= height of tree)
    - Means it is a balanced tree



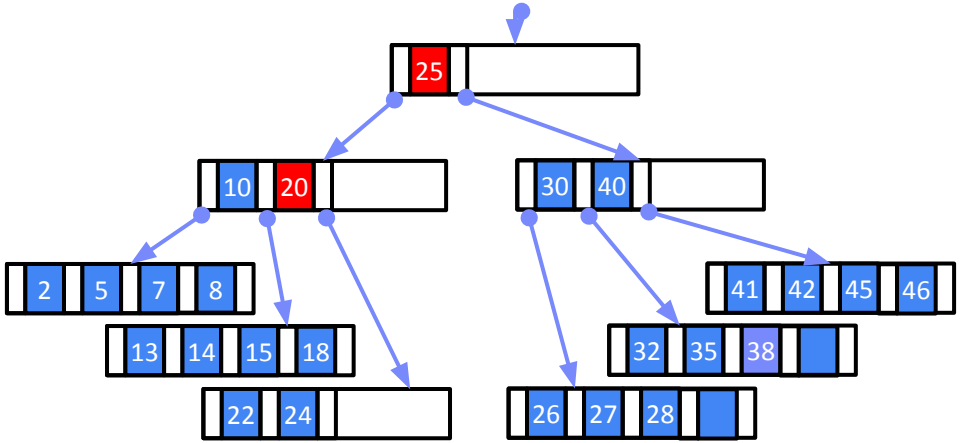
B-Tree Search

- **Example B-Tree k=2**
  - Get **38** -> D38



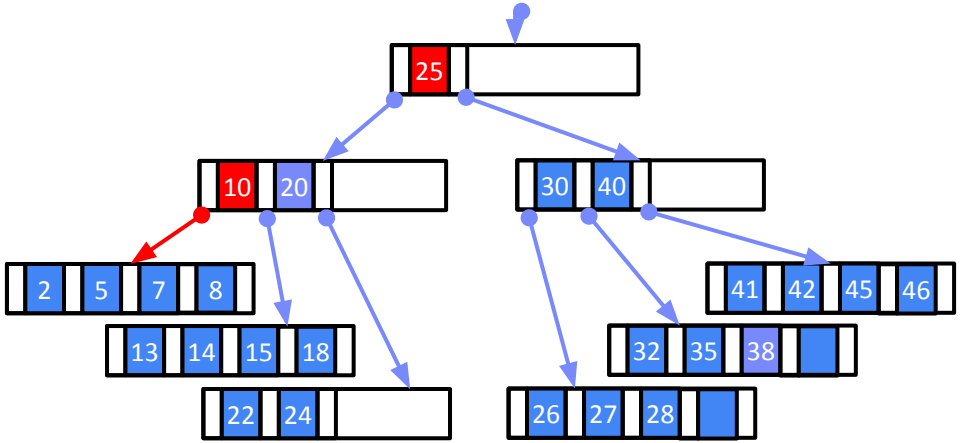
B-Tree Search

- **Example B-Tree k=2**
  - Get **38** -> D38
  - Get **20** -> D20



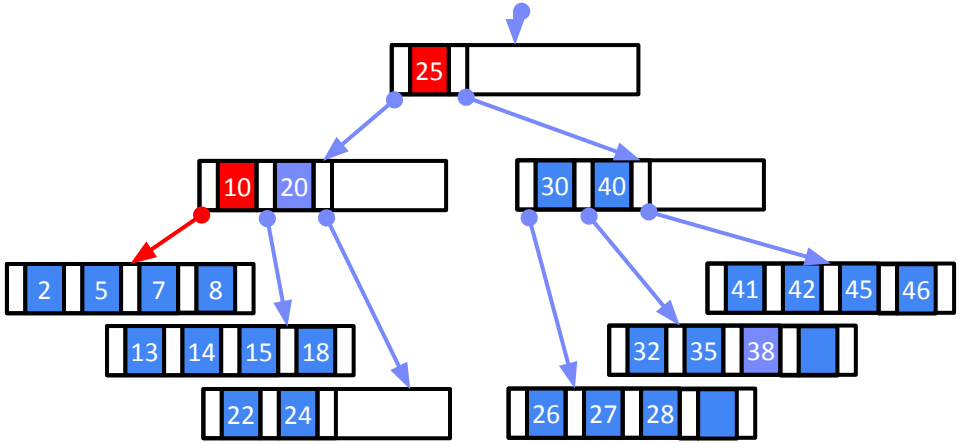
B-Tree Search

- **Example B-Tree k=2**
  - Get **38** -> D38
  - Get **20** -> D20
  - Get **6** -> NULL



B-Tree Search

- **Example B-Tree k=2**
  - Get **38** -> D38
  - Get **20** -> D20
  - Get **6** -> NULL



- **Lookup Q_K within a node**
  - Scan / binary search keys for Q_K, if $K_i=Q_K$, return D_i
  - If node does not contain key
    - If leaf node, abort search w/ NULL (not found), otherwise
    - Decent into subtree Pi with $K_i < Q_K \leq K_{i+1}$

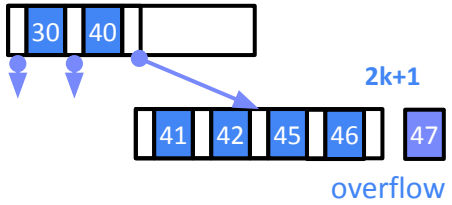
B-Tree Insert

- **Basic Insertion Approach**
  - **Always insert into leaf nodes!**
  - Find position similar to **lookup algorithm**, insert and maintain sorted order
  - If node overflows (exceeds 2k entries) □ **node splitting**

B-Tree Insert

- **Basic Insertion Approach**
  - **Always insert into leaf nodes!**
  - Find position similar to lookup, insert and maintain sorted order
  - If node overflows (exceeds 2k entries) □ **node splitting**

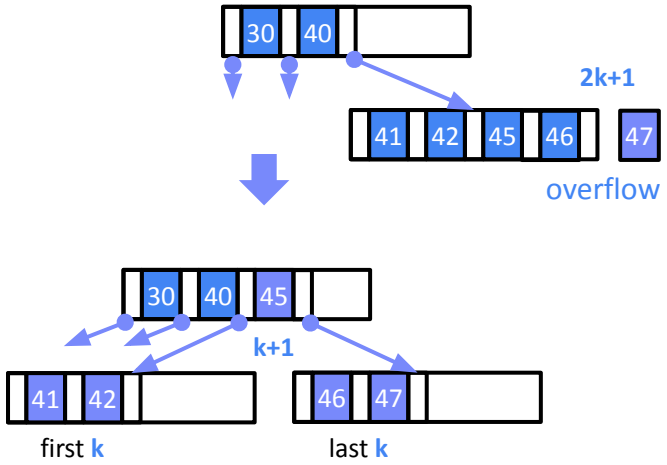
- **Node Splitting Approach**
  - Split the **2k+1** entries into two leaf nodes
  - **Left node:** first k entries
  - **Right node:** last k entries
  - (k+1)th entry inserted into parent node
    - □ can cause **recursive splitting**
  - Height stays the same
    - Special case: root split (h++)



B-Tree Insert

- **Basic Insertion Approach**
  - **Always insert into leaf nodes!**
  - Find position similar to lookup, insert and maintain sorted order
  - If node overflows (exceeds 2k entries) □ **node splitting**

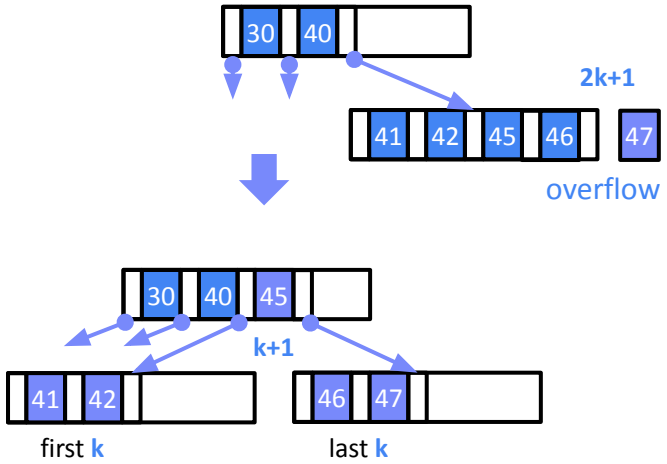
- **Node Splitting Approach**
  - Split the **2k+1** entries into two leaf nodes
  - **Left node:** first k entries
  - **Right node:** last k entries
  - (k+1)th entry inserted into parent node
    - □ can cause **recursive splitting**
  - Height stays the same
    - Special case: root split (h++)



B-Tree Insert

- **Basic Insertion Approach**
  - **Always insert into leaf nodes!**
  - Find position similar to lookup, insert and maintain sorted order
  - If node overflows (exceeds 2k entries) □ **node splitting**

- **Node Splitting Approach**
  - Split the **2k+1** entries into two leaf nodes
  - **Left node:** first k entries
  - **Right node:** last k entries
  - (k+1)th entry inserted into parent node
    - □ can cause **recursive splitting**
  - Height stays the same
    - Special case: root split (h++)



- **B-Tree is self-balancing**

B-Tree Insert, cont. (Example w/ k=1)

- Insert 1



B-Tree Insert, cont. (Example w/ k=1)

- Insert 1



- Insert 5



B-Tree Insert, cont. (Example w/ k=1)

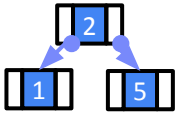
- Insert 1



- Insert 5



- Insert 2 (split)



B-Tree Insert, cont. (Example w/ k=1)

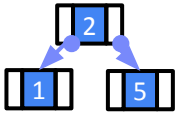
- Insert 1



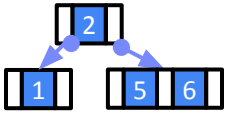
- Insert 5



- Insert 2 (split)



- Insert 6



B-Tree Insert, cont. (Example w/ k=1)

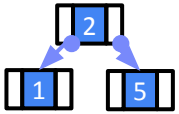
- Insert 1



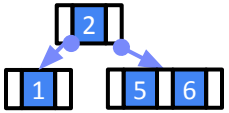
- Insert 5



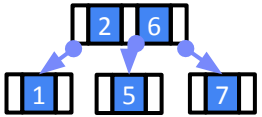
- Insert 2 (split)



- Insert 6



- Insert 7 (split)



B-Tree Insert, cont. (Example w/ k=1)

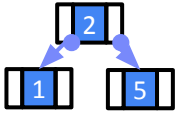
- Insert 1



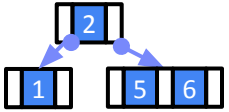
- Insert 5



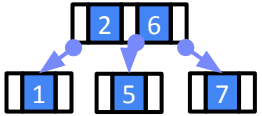
- Insert 2 (split)



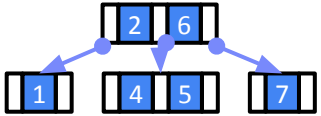
- Insert 6



- Insert 7 (split)



- Insert 4



B-Tree Insert, cont. (Example w/ k=1)

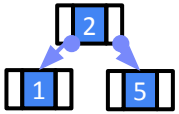
- Insert 1



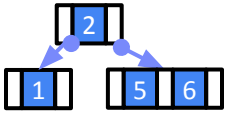
- Insert 5



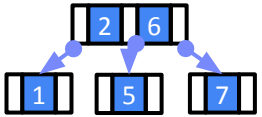
- Insert 2 (split)



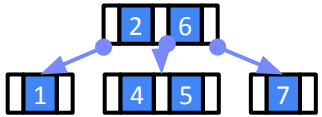
- Insert 6



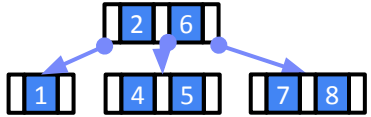
- Insert 7 (split)



- Insert 4



- Insert 8



B-Tree Insert, cont. (Example w/ k=1)

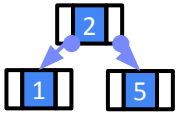
- Insert 1



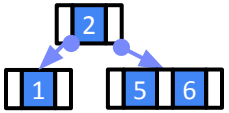
- Insert 5



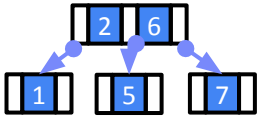
- Insert 2 (split)



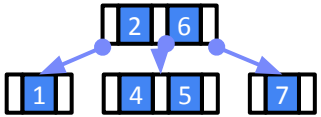
- Insert 6



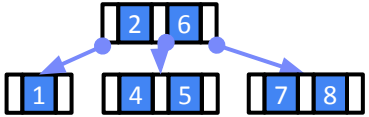
- Insert 7 (split)



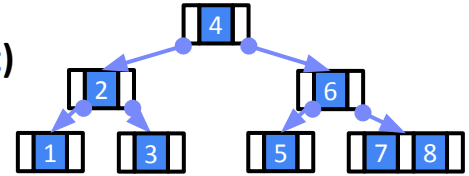
- Insert 4



- Insert 8

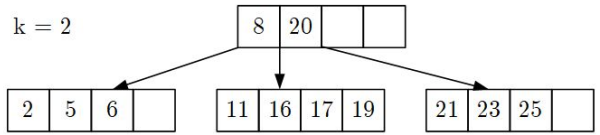


- Insert 3 (2x split)



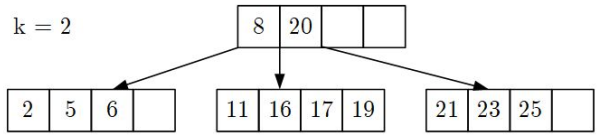
Practice Insert

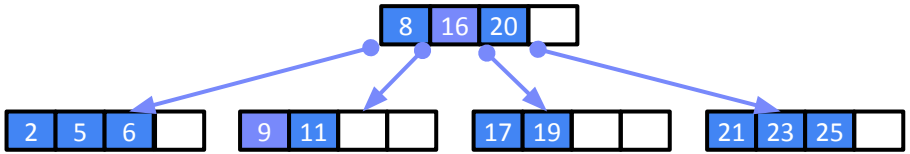
- Given B-tree below, insert key 9 and draw resulting B-tree



Practice Insert

- Given B-tree below, insert key 9 and draw resulting B-tree





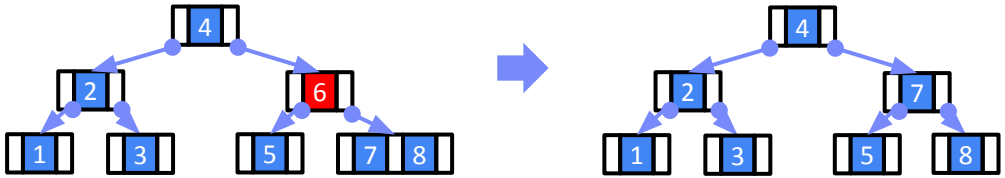
B-Tree Delete

- **Basic Deletion Approach**
  - Lookup deletion key, abort if non-existing
  - Case inner node: **move entry** from fullest successor node into position
  - Case leaf node: if underflows (<k entries) **-> merge w/ sibling**

B-Tree Delete

- **Basic Deletion Approach**
  - Lookup deletion key, abort if non-existing
  - Case inner node: **move entry** from fullest successor node into position
  - Case leaf node: if underflows (<k entries) **-> merge w/ sibling**

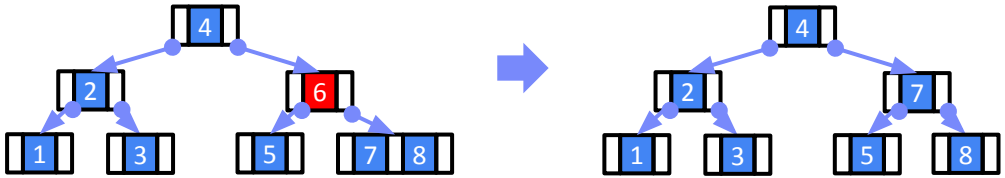
- **Example (k=1)**
  - Case inner



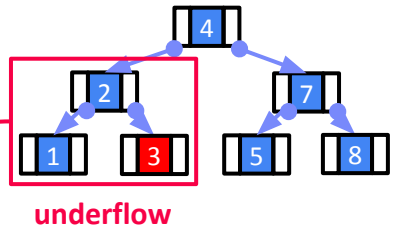
B-Tree Delete

- **Basic Deletion Approach**
  - Lookup deletion key, abort if non-existing
  - Case inner node: **move entry** from fullest successor node into position
  - Case leaf node: if underflows (<k entries) **-> merge w/ sibling**

- **Example (k=1)**
  - Case inner



  - Case leaf

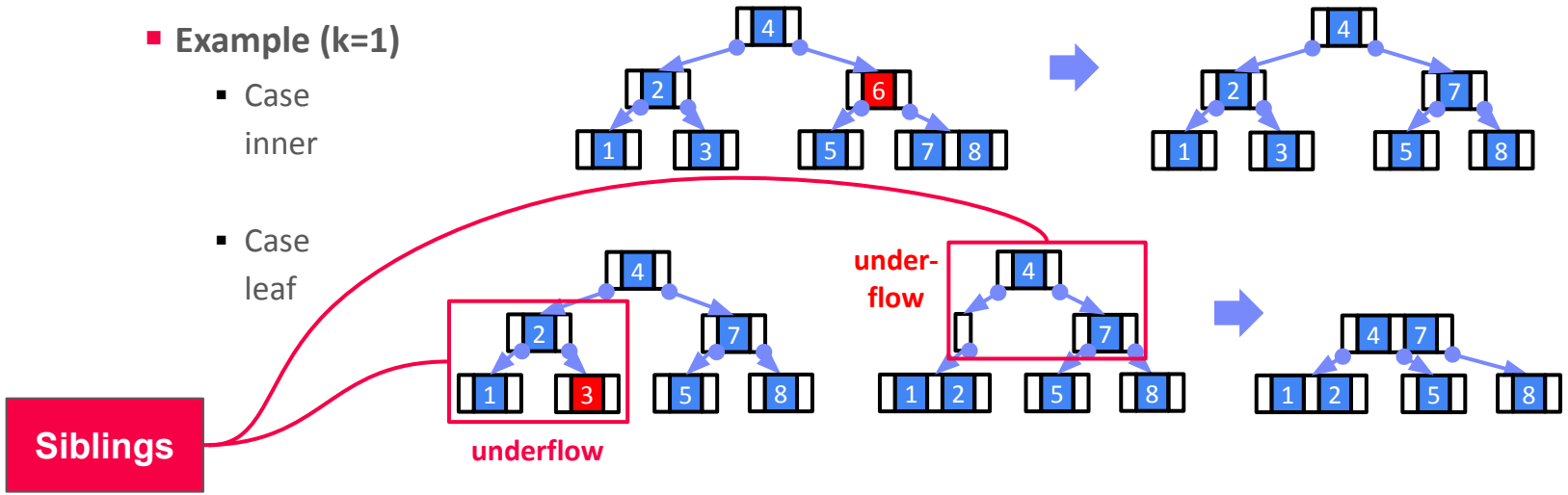


Siblings

B-Tree Delete

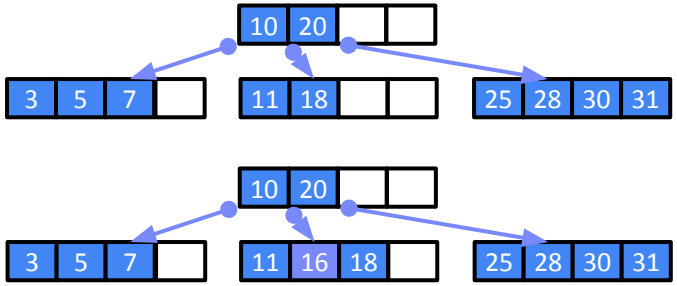
- **Basic Deletion Approach**
  - Lookup deletion key, abort if non-existing
  - Case inner node: **move entry** from fullest successor node into position
  - Case leaf node: if underflows (<k entries) **-> merge w/ sibling**

- **Example (k=1)**
  - Case inner
  - Case leaf



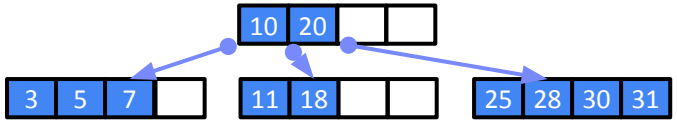
B-Tree Insert and Delete w/ k=2

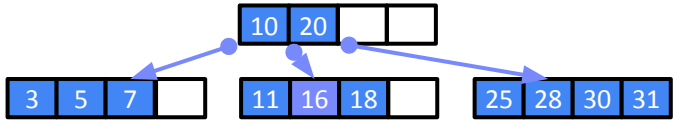
- **Insert/Delete Examples**
  - Original
  - Insert 16

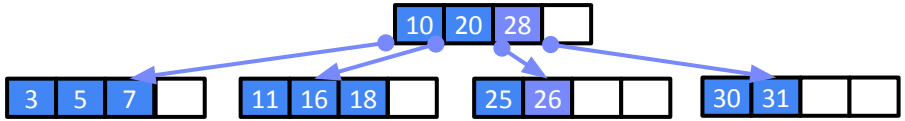


B-Tree Insert and Delete w/ k=2

- **Insert/Delete Examples**
  - Original
  - Insert 16
  - Insert 26 (split)

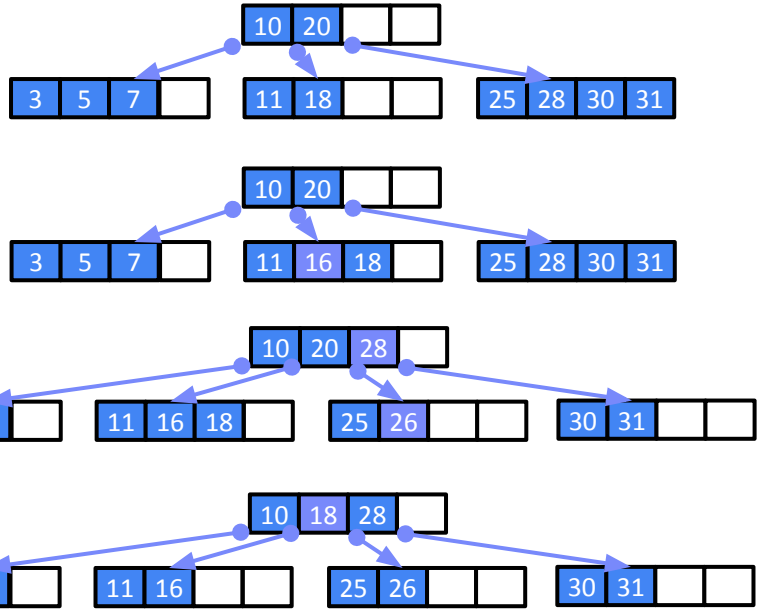






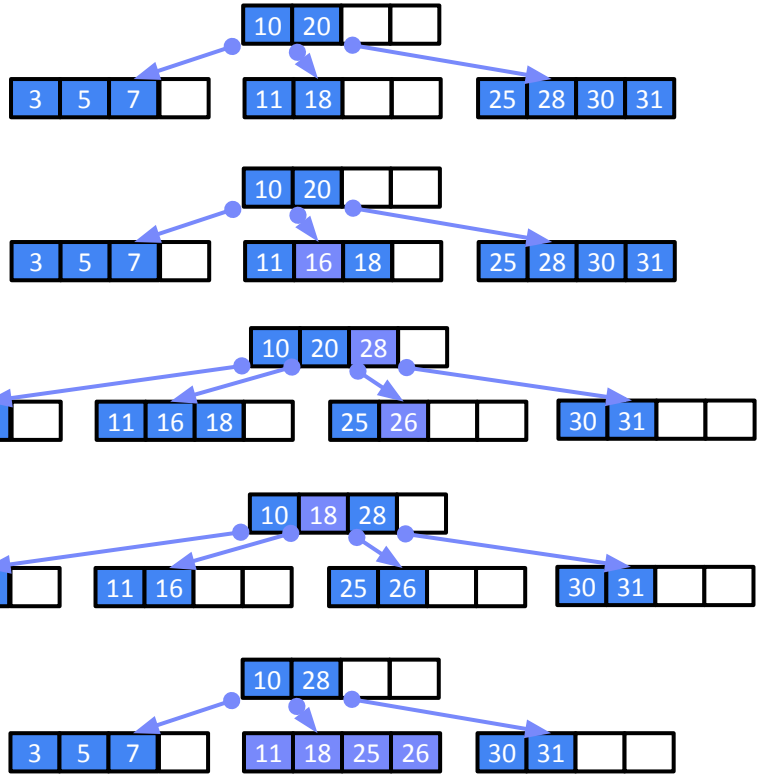
B-Tree Insert and Delete w/ k=2

- **Insert/Delete Examples**
  - Original
  - Insert 16
  - Insert 26 (split)
  - Delete 20 (inner node)



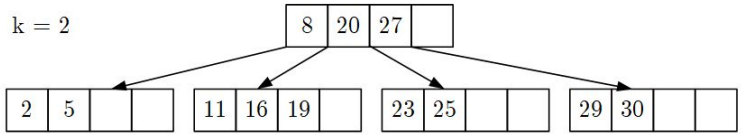
B-Tree Insert and Delete w/ k=2

- **Insert/Delete Examples**
  - Original
  - Insert 16
  - Insert 26 (split)
  - Delete 20 (inner node)
  - Delete 16 (under flow)



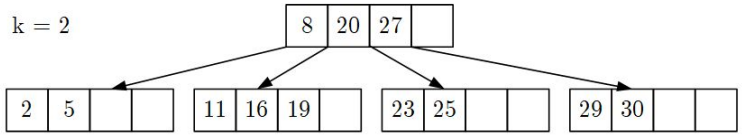
Practice (at home)

- Given B-tree below, delete key 27, and draw resulting B-tree

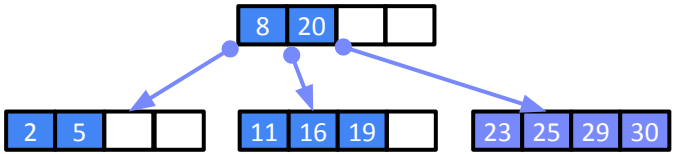


Practice (at home)

- Given B-tree below, delete key 27, and draw resulting B-tree

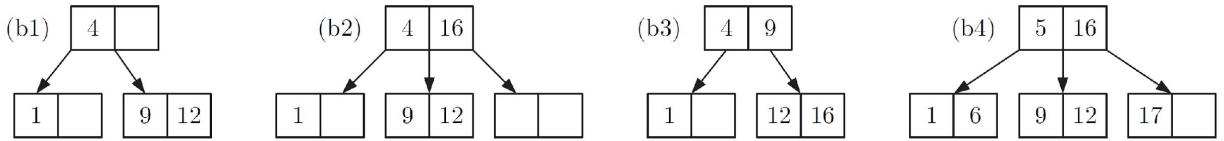


□ Inner node, we move up 29, which creates an underflow in the leaf-node □ merge w/ siblings (remember, sometimes multiple solutions possible)



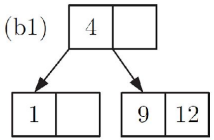
Further Practice (at home)

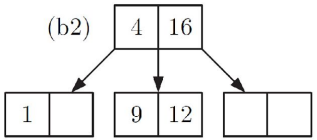
- Which of the following trees are valid – i.e., satisfy the constraints of B-trees with k=1? Mark each tree as valid or invalid and name the violations



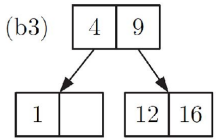
Further Practice (at home)

- **Which of the following trees are valid – i.e., satisfy the constraints of B-trees with k=1? Mark each tree as valid or invalid and name the violations**

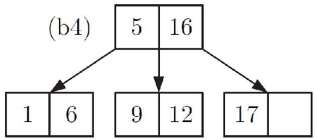




(empty leaf node, **underflow**)



(**invalid #** of pointers and subtrees)



(**invalid ordering** of data items, 6>5 but in left subtree)

Materialized Views
(quick overview)

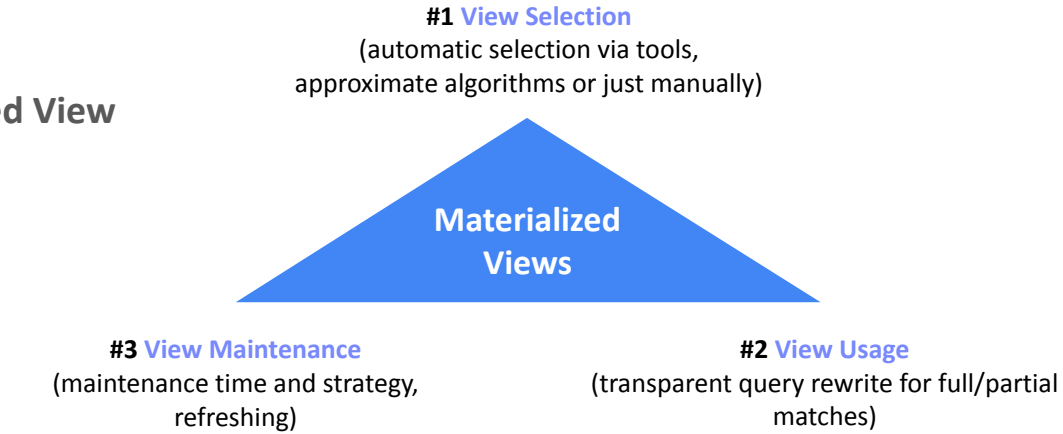
Overview Materialized Views

- **Core Idea of Views**
  - Identify frequently recurring queries and expose them as a virtual table.
  - **Normal views** store the query definition, not the query results.
  - **Materialized views** precompute query results, store them physically in the database, and allow reuse.
  - **Materialized views** can improve performance but may become stale until refreshed.

Overview Materialized Views

- **Core Idea of Views**
  - Identify frequently recurring queries and expose them as a virtual table.
  - **Normal views** store the query definition, not the query results.
  - **Materialized views** precompute query results, store them physically in the database, and allow reuse.
  - **Materialized views** can improve performance but may become stale until refreshed.

- **The Materialized View Lifecycle**



Materialized Views in PostgreSQL

- **View Selection**
  - **Manual definition** of materialized view

```
CREATE MATERIALIZED VIEW TopScorer AS
  SELECT P.Name, Count(*)
    FROM Players P, Goals G
    WHERE P.Pid=G.Pid AND G.GOwn=FALSE
    GROUP BY P.Name
    ORDER BY Count(*) DESC;
```

Materialized Views in PostgreSQL

- **View Selection**
  - **Manual definition** of materialized view
- **View Usage**
  - **Manual use** of view
  - Just use it in your queries
- **View Maintenance**
  - **Manual refresh**
  - Data is deleted and fetched again

```
CREATE MATERIALIZED VIEW TopScorer AS
  SELECT P.Name, Count(*)
    FROM Players P, Goals G
    WHERE P.Pid=G.Pid AND G.GOwn=FALSE
    GROUP BY P.Name
    ORDER BY Count(*) DESC;
```

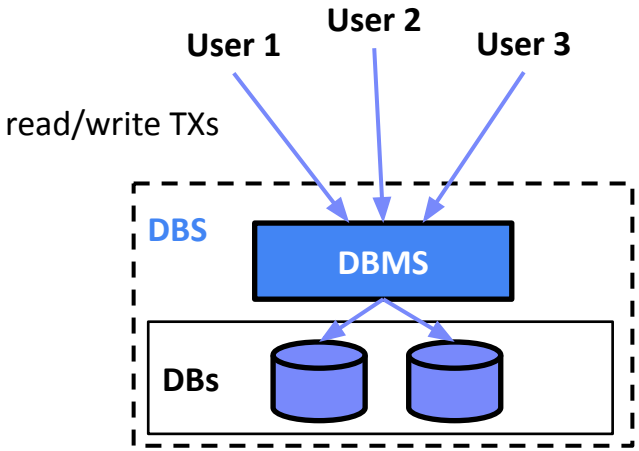
```
REFRESH MATERIALIZED VIEW TopScorer;
```

Name	Count
James Rodríguez	6
Thomas Müller	5
Robin van Persie	4
Neymar	4
Lionel Messi	4
Arjen Robben	3

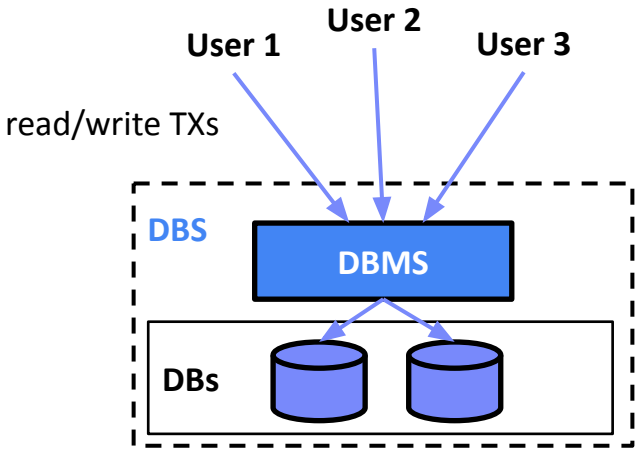
Transaction Processing
(quick overview)

[More details → Course Data Integration and Large-Scale Analysis]

Transaction (TX) Processing



Transaction (TX) Processing



#1 Multiple users / concurrency
□ Correctness?

#2 Various failures
□ Reliability?

Deadlocks
Constraint violations
Network failure
Crash/power failure
Disk failure

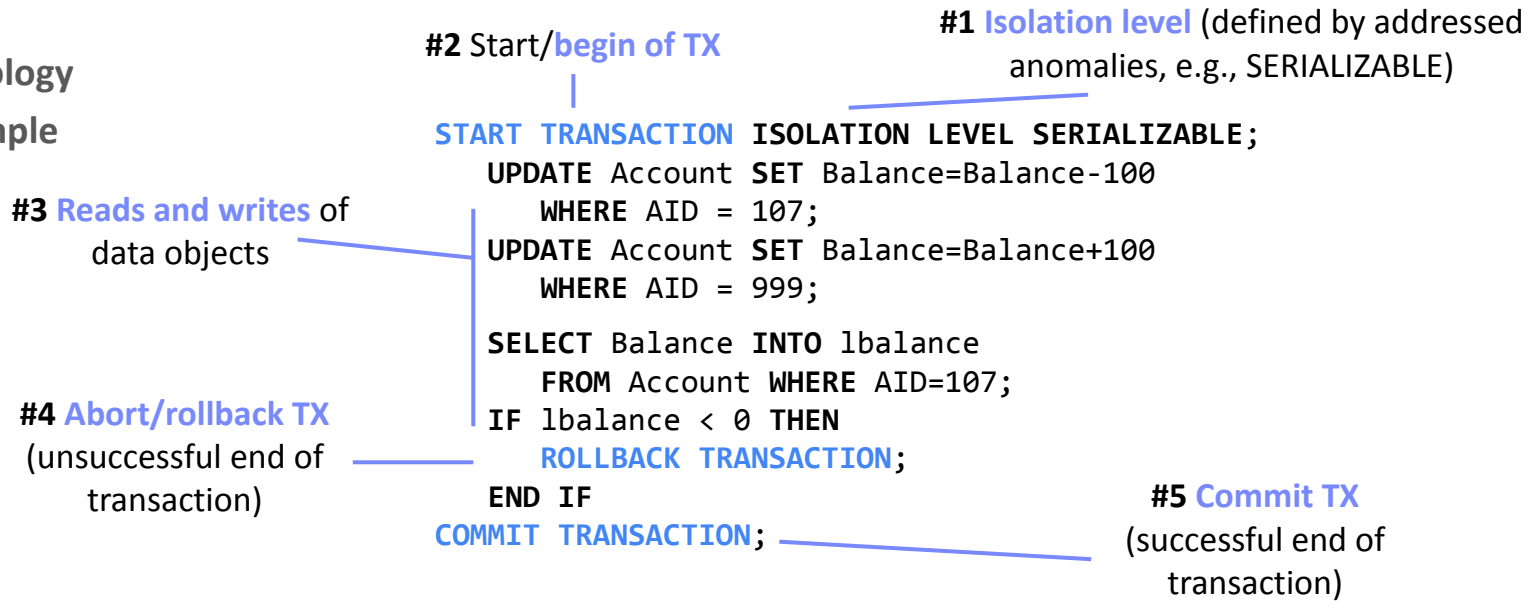
Terminology of Transactions

- **Database Transaction**
  - A transaction (TX) is a **series of steps** that brings a database from a **consistent state** into another (not necessarily different) **consistent state**
  - **ACID properties** (atomicity, consistency, isolation, durability)

Terminology of Transactions

- **Database Transaction**
  - A transaction (TX) is a **series of steps** that brings a database from a **consistent state** into another (not necessarily different) **consistent state**
  - **ACID properties** (atomicity, consistency, isolation, durability)

- **Terminology by Example**



ACID Properties

[Theo Härder, Andreas Reuter: Principles of Transaction-Oriented Database Recovery. **ACM Comput. Surv. 15(4) 1983**]



- **Atomicity**
  - A transaction is executed atomically (**completely or not at all**)
  - If the transaction fails/aborts no changes are made to the database

ACID Properties

[Theo Härder, Andreas Reuter: Principles of Transaction-Oriented Database Recovery. **ACM Comput. Surv. 15(4) 1983**]



- **Atomicity**
  - A transaction is executed atomically (**completely or not at all**)
  - If the transaction fails/aborts no changes are made to the database
- **Consistency**
  - A successful transaction ensures that all **consistency constraints are met** (referential integrity, semantic/domain constraints)

ACID Properties

[Theo Härder, Andreas Reuter: Principles of Transaction-Oriented Database Recovery. **ACM Comput. Surv. 15(4) 1983**]



- **Atomicity**
  - A transaction is executed atomically (**completely or not at all**)
  - If the transaction fails/aborts no changes are made to the database
- **Consistency**
  - A successful transaction ensures that all **consistency constraints are met** (referential integrity, semantic/domain constraints)
- **Isolation**
  - Concurrent transactions are executed in isolation of each other
  - **Appearance of serial transaction execution**

ACID Properties

[Theo Härder, Andreas Reuter: Principles of Transaction-Oriented Database Recovery. **ACM Comput. Surv. 15(4) 1983**]



- **Atomicity**
  - A transaction is executed atomically (**completely or not at all**)
  - If the transaction fails/aborts no changes are made to the database
- **Consistency**
  - A successful transaction ensures that all **consistency constraints are met** (referential integrity, semantic/domain constraints)
- **Isolation**
  - Concurrent transactions are executed in isolation of each other
  - **Appearance of serial transaction execution**
- **Durability**
  - **Guaranteed persistence** of all changes made by a successful transaction
  - In case of system failures, the database is recoverable

Example of Anomaly: Lost Update

TA1 updates points for Exercise 1

```
SELECT Pts INTO :points
   FROM Students WHERE Sid=789;

points += 23.5;

UPDATE Students SET Pts=:points
   WHERE Sid=789;
COMMIT TRANSACTION;
```

TA2 updates points for Exercise 2 (with slight delay)

```
SELECT Pts INTO :points
   FROM Students WHERE Sid=789;

points += 24.0;

UPDATE Students SET Pts=:points
   WHERE Sid=789;
COMMIT TRANSACTION;
```



Time

Example of Anomaly: Lost Update

TA1 updates points for Exercise 1

```
SELECT Pts INTO :points
   FROM Students WHERE Sid=789;

points += 23.5;

UPDATE Students SET Pts=:points
   WHERE Sid=789;
COMMIT TRANSACTION;
```

TA2 updates points for Exercise 2 (with slight delay)

```
SELECT Pts INTO :points
   FROM Students WHERE Sid=789;

points += 24.0;

UPDATE Students SET Pts=:points
   WHERE Sid=789;
COMMIT TRANSACTION;
```

Time



Student received 24 instead of 47.5 points
(lost update 23.5)

- **Problem:** Write-write dependency
- **Solution:** Use isolation level SERIALIZABLE or LOCK TABLE Students WRITE;

Conclusions

- **Summary**
  - **Physical Design and Tuning**
    - **Compression (a bit)**
    - **Index**
    - **B-trees**
  - **Transaction processing (a bit)**
- **Next**
  - **(6) Mon. 27.04.2025:** Physical design and tuning
  - **No lecture on 04.05.2026** (time for Ex 2 + **VO exam #2**)
  - **(7) Mon. 11.05.2026:** Database APIs + Python + Exercise 3